
Session 4 — Linear Algebra II

What today is about — the payoff session

Today the two halves of the course meet.

- *Eigenvectors* and *eigenvalues*: the special directions of a transformation.
- **Principal Component Analysis**: the flagship tool for reducing and visualising high-dimensional data.
- The *SVD*: the general engine behind PCA.
- *Dynamics*: how eigenvalues decide whether a system settles, oscillates, or explodes.

We build slowly, with a picture at every step.

Key idea

The **covariance matrix** from Session 2 (a statistical object) read through its **eigenvectors** (a linear-algebra tool) *is* Principal Component Analysis. That single sentence is the destination of today.

So keep the covariance matrix in mind as we build the eigenvalue machinery — we are assembling exactly the pieces PCA needs.

Recall: a matrix moves vectors around

From last session: multiplying a vector by a matrix A *transforms* it — it usually lands pointing in a new direction.

Today's question is simple and powerful:

*Are there any special directions that the matrix does **not** turn?*

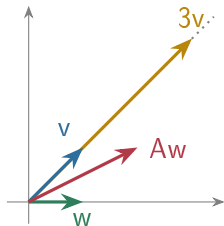
The answer — yes, a few — unlocks PCA and the behaviour of dynamical systems. Let us just *look* at what a matrix does to a handful of vectors.

Watch what a matrix does to a few vectors

Take $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ and apply it:

- $A\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$ — **turned** off its line.
- $A\begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \end{pmatrix} = 3\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ — **stayed** on its line, just $3\times$ longer!

The direction $(1, 1)$ is *special*: A only stretches it.



The special directions have a name

Definition

A nonzero vector v is an **eigenvector** of A if A only *stretches* it, without changing its direction:

$$Av = \lambda v,$$

where the number λ is the **eigenvalue** — the stretch factor.

In the example, $(1, 1)$ is an eigenvector with eigenvalue $\lambda = 3$.

Eigenvectors are the “grain of the wood”

Intuition

Think of the transformation as flowing space around. Almost every arrow gets rotated as well as stretched. But along an eigenvector, the transformation is pure stretch: the arrow stays on its own line, only getting longer, shorter, or flipped.

Along these special directions a complicated matrix behaves like plain multiplication by a number. That simplicity is what makes eigenvectors so powerful.

How do we *find* the special directions?

We want $Av = \lambda v$ for some nonzero v . Rearrange:

$$Av - \lambda v = 0 \implies (A - \lambda I)v = 0.$$

Intuition

A matrix that sends a *nonzero* vector to 0 must **collapse space** — and from last session, that means its determinant is zero. So the special stretch factors λ are exactly the ones with

$$\det(A - \lambda I) = 0 \quad (\text{the } \textit{characteristic equation}).$$

Finding them — worked example

Example

For $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, solve $\det(A - \lambda I) = 0$:

$$\det \begin{pmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{pmatrix} = (2 - \lambda)^2 - 1 = 0 \implies 2 - \lambda = \pm 1 \implies \lambda = 3 \text{ or } 1.$$

The stretch factors are 3 and 1. Their directions come from $(A - \lambda I)v = 0$: they are $(1, 1)$ for $\lambda = 3$ and $(1, -1)$ for $\lambda = 1$ — and they are *orthogonal*.

Two free checks on your eigenvalues

Useful fact

For any square matrix:

$$\sum_i \lambda_i = \text{tr}(A) \text{ (sum of the diagonal),} \quad \prod_i \lambda_i = \det(A).$$

For the example above: $\lambda_1 + \lambda_2 = 3 + 1 = 4 = \text{tr}(A)$, and $\lambda_1 \lambda_2 = 3 \cdot 1 = 3 = \det(A)$.
Always use these to catch arithmetic slips before going any further.

Why eigenvalues make repeated action easy

Intuition

Along an eigenvector, applying A just multiplies by λ . So applying A *again and again* multiplies by λ each time: after k steps, the component is scaled by λ^k .

Formally, if we collect the eigenvectors and eigenvalues into $A = PDP^{-1}$, then $A^k = PD^kP^{-1}$ with $D^k = \text{diag}(\lambda_1^k, \dots)$. This is why the eigenvalues alone govern the long-run behaviour — exactly what we need for dynamics later.

Symmetric matrices: perpendicular axes

Key idea

If $A = A^T$ (symmetric — as *every* covariance matrix is), then its eigenvalues are real and its eigenvectors are mutually *orthogonal* (perpendicular). The transformation is then just independent stretches along a set of perpendicular axes.

This clean structure — perpendicular directions, each with its own stretch — is precisely what makes PCA work.

A friendly special case: positive-definite

Definition

A symmetric matrix is **positive-definite** when all its eigenvalues are *positive*.

Intuition

Positive eigenvalues mean the matrix *stretches outward* along every axis — it never flips a direction or collapses it. Eigenvalues 3 and 1: positive-definite. Eigenvalues 3 and -1 : *not* (one direction gets flipped).

Covariance matrices are always positive-semidefinite (eigenvalues ≥ 0), because each eigenvalue is an *amount of variance* — and variance can never be negative. That is why PCA's “variance along a component” is never negative.

Practice — try it

Let $A = \begin{pmatrix} 3 & 0 \\ 0 & -2 \end{pmatrix}$ (already diagonal).

1. What are its eigenvalues and eigenvectors?
2. Check the trace and the determinant against your eigenvalues.
3. What does A do to the vector $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$?

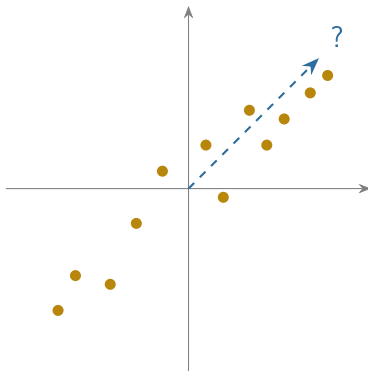
PCA starts with a picture

Suppose two measurements are correlated, so the data form a tilted, cigar-shaped cloud.

Intuition

If you had to keep just *one* number per point — one direction to measure along — which direction loses the least information?

The *long axis* of the cloud. PCA finds that direction automatically.



The problem PCA solves

High-dimensional data are hard to see and often redundant. Because many measured variables are correlated, the cloud of data points really lies close to a lower-dimensional subspace.

PCA finds the few directions that capture the most variation, so we can describe, visualise, and compress the data with little loss. It answers: *along which directions does the data actually vary?*

Key idea

The principal components are the **eigenvectors of the covariance matrix**, and each eigenvalue is the **amount of variance** along that direction. The first component is the direction of greatest spread; the next is the greatest remaining spread orthogonal to it; and so on.

Everything from the first half of today — eigenvectors, symmetric matrices, orthogonality — comes together in this one idea.

The PCA recipe

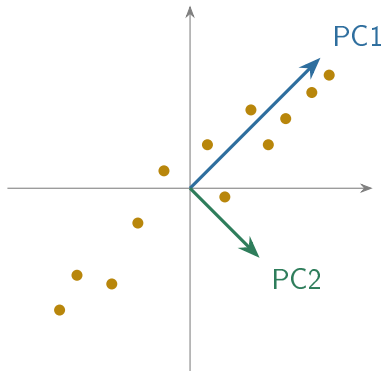
1. **Centre** the data: subtract each variable's mean, so the cloud sits at the origin.
2. Form the **covariance matrix** C of the centred data.
3. Compute the **eigenvectors and eigenvalues** of C .
4. Order the eigenvectors by eigenvalue, largest first: these are principal components $1, 2, 3, \dots$
5. **Project** the data onto the top few components to reduce dimension.

What PCA is doing, geometrically

Intuition

PCA finds the *long axis* of the cloud (most variance — PC1) and the short axis perpendicular to it (PC2). Rotating onto these axes “straightens” the cloud so its directions become uncorrelated.

Keeping only the long axis is dimensionality reduction: describe each point by its position along the direction that matters most.



Example

Two mean-centred variables have covariance matrix $C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$.

We already found its eigenpairs: $\lambda_1 = 3$ along $(1, 1)$, and $\lambda_2 = 1$ along $(1, -1)$. So the first principal component points along $(1, 1)$ (the two variables rise together) and carries variance 3; the second carries variance 1. The fraction of variance explained by the first component is

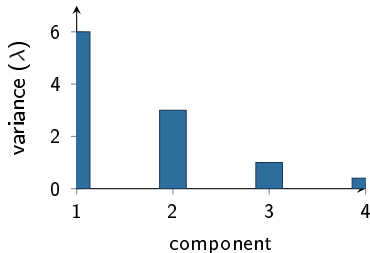
$$\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{3}{4} = 75\%.$$

Reading the eigenvalues

Useful fact

Total variance = $\sum_i \lambda_i = \text{tr}(\mathbf{C})$. The proportion explained by component j is $\lambda_j / \sum_i \lambda_i$. All $\lambda_i \geq 0$ (they are variances).

Plot the eigenvalues in decreasing order — the **scree plot**. The “elbow” tells you how many components are worth keeping.



Keep the first two — the rest add little.

A caution about PCA

Common pitfall

PCA is a *linear* method and is sensitive to *scaling*. A variable measured in large units (say reaction time in milliseconds) can dominate the components purely because of its units. Standardising the variables, or using the correlation matrix, is often necessary.

Also remember: PCA finds directions of *variance*, which are not always the directions that matter for a task. Variance is not the same as relevance.

Link to cognitive science

PCA reduces recordings of hundreds of neurons to a few “latent” dimensions along which population activity evolves (neural state-space analysis); it compresses fMRI voxels and EEG channels; it extracts the principal components of face images (“eigenfaces”); and it summarises questionnaire batteries into a few underlying factors.

Whenever someone says a system is “effectively low-dimensional”, PCA is usually how they found out.

Practice — try it

After PCA on some neural data, the covariance has eigenvalues $\lambda = (6, 3, 1)$ along three orthogonal components.

1. What is the total variance?
2. What fraction does the first component explain?
3. How many components are needed to keep at least 85% of the variance?

The SVD: the general engine behind PCA

Definition

The **Singular Value Decomposition** writes *any* matrix as $X = U\Sigma V^T$: a rotation, a set of non-negative stretches (the *singular values* in Σ), and another rotation.

The idea to carry away: *every* linear transformation is a rotation, then a scaling along perpendicular axes, then another rotation. PCA is just the SVD applied to the centred data matrix. (You do not need the details today — just the picture.)

Eigenvalues as growth rates: the one-number version

Start with a single number and multiply it by the same factor over and over:

- factor 0.5: $1 \rightarrow 0.5 \rightarrow 0.25 \rightarrow 0.125 \rightarrow \dots \rightarrow 0$ (**decays**).
- factor 1.2: $1 \rightarrow 1.2 \rightarrow 1.44 \rightarrow 1.73 \rightarrow \dots \rightarrow \infty$ (**grows**).

Intuition

An eigenvalue is exactly this repeated factor, but along an eigenvector direction of a matrix. So $|\lambda| < 1$ means that direction dies away; $|\lambda| > 1$ means it blows up.

Eigenvalues predict the future of a system

Many models describe how a state x evolves, step by step or continuously.

Key idea

Discrete time $x_{t+1} = Ax_t$: stable (shrinks to zero) if every $|\lambda| < 1$; blows up if some $|\lambda| > 1$.

Continuous time $\dot{x} = Ax$: stable if every eigenvalue has *negative real part*; imaginary parts produce oscillations.

Why eigenvalues govern stability

Intuition

Decompose any starting state into its eigenvector components; each then evolves independently, growing or decaying at its own rate. The largest eigenvalue eventually dominates and sets the long-run behaviour.

This is why the eigenvalues alone tell you whether a dynamical system settles, oscillates, or explodes — without solving anything in detail.

Example

A discrete system $x_{t+1} = Ax_t$ has eigenvalues $\lambda_1 = 0.5$ and $\lambda_2 = 1.2$. What happens over time?

Along the first eigenvector the state is multiplied by 0.5 each step, so that component decays to zero. Along the second it is multiplied by 1.2, growing by 20% each step. Because one $|\lambda| > 1$, the system is **unstable**: almost every starting state eventually grows without bound. It would be stable only if *both* eigenvalues had magnitude below 1.

Link to cognitive science

Recurrent neural networks and models of neural population dynamics are exactly systems of the form $\dot{x} = Ax$ (plus inputs and nonlinearities). Whether a memory persists, an activity pattern decays, or a network oscillates is governed by the eigenvalues of its connectivity matrix.

This also closes the loop with the differential-equations material elsewhere in the programme: the qualitative behaviour of a linear system is read straight off its eigenvalues.

Practice — try it

Stable or unstable?

1. A continuous system with eigenvalues -1 and -3 .
2. A continuous system with eigenvalues -2 and $+0.5$.
3. A discrete system with eigenvalues 0.9 and 0.4 .

Session 4 — what we built

- *Eigenvectors* are the directions a matrix only stretches, by factor λ ; symmetric matrices have real eigenvalues and orthogonal eigenvectors.
- *PCA* uses the eigenvectors of the covariance matrix as principal components, and the eigenvalues as the variances along them.
- The *SVD* is the general rotation–scale–rotation behind PCA.
- In *dynamics*, eigenvalues set the growth or decay of each mode, and thus stability.

Next: probability again — now armed with vectors and matrices.